

ОБЛАЧНАЯ НАТИВНАЯ ЭКОСИСТЕМА

Введение: понимание архитектуры программного обеспечения и систем

Существует множество способов организации приложений в логические компоненты (программная архитектура), которые затем размещаются на разных компьютерах (системная архитектура).

5 декабря 2019 года, 10:17 [Кэтрин Паганини](#)



Изображение предоставлено [giovanni gargiulo](#) с [Pixabay](#).

Этот пост является частью [продолжающейся серии](#) статей Кэтрин Паганини, посвященных объяснению IT-концепций для руководителей бизнеса. Некоторые ключевые понятия, о которых говорится в этой статье, объясняются в ее предыдущей статье [«Основы распределенных систем»](#).

Поскольку технологии становятся ключевым фактором дифференциации, руководители компаний все чаще ведут стратегические переговоры с ИТ-руководителями, чтобы лучше понять, как разрабатываемые ими решения будут соответствовать потребностям бизнеса. На этом этапе крайне важно, бы у них было базовое понимание того, как работают программное обеспечение и системная архитектура, а также того, как решения,

масштабируемости, гибкости и т. д. мы надеемся, что этот обзор на высоком уровне даст необходимый контекст для таких переговоров. Этот подход будет особенно интересен системным архитекторам, которым необходимо понимать, развертывать и администрировать распределенные облачные системы.

Как отмечалось в нашей предыдущей статье распределенные системы — это сложные программные продукты. Чтобы справиться с этой сложностью, системы должны быть правильно организованы. Для начала давайте разберемся в двух ключевых понятиях: архитектуре программного обеспечения и системной архитектуре.

Архитектура программного обеспечения — это логическая организация распределенной системы на программные компоненты. Вместо одного большого монолитного приложения распределенные системы разбиваются на несколько компонентов. То, как эти компоненты разбиваются на части, влияет на все аспекты системы — от производительности до надежности и задержки отклика.

Архитектура системы — это размещение программных компонентов на физических машинах. Два тесно связанных компонента могут быть размещены на одной машине или на разных. Расположение компонентов также влияет на производительность и надежность.

В конечном итоге архитектурный стиль определяет, как соединяются компоненты, как происходит обмен данными и как все они работают вместе как единая система.

Архитектура программного обеспечения

Существует множество различных архитектурных стилей, в том числе многоуровневые, объектно-ориентированные, сервис-ориентированные архитектуры, архитектуры RESTful, архитектуры «издатель — подписчик» и так далее. Выбор стиля в конечном итоге зависит от приложения. Кроме того, по мере создания более сложных взаимосвязанных систем распределенные системы становятся все больше. В результате такие процессы, как коммуникация, должны быть упрощены и оптимизированы.

Многоуровневая архитектура

нисходящие вызовы (отправляют запросы на нижний уровень). Компоненты нижнего уровня могут выполнять восходящие вызовы (отправлять запросы вверх), но обычно они отвечают только на запросы верхнего уровня.

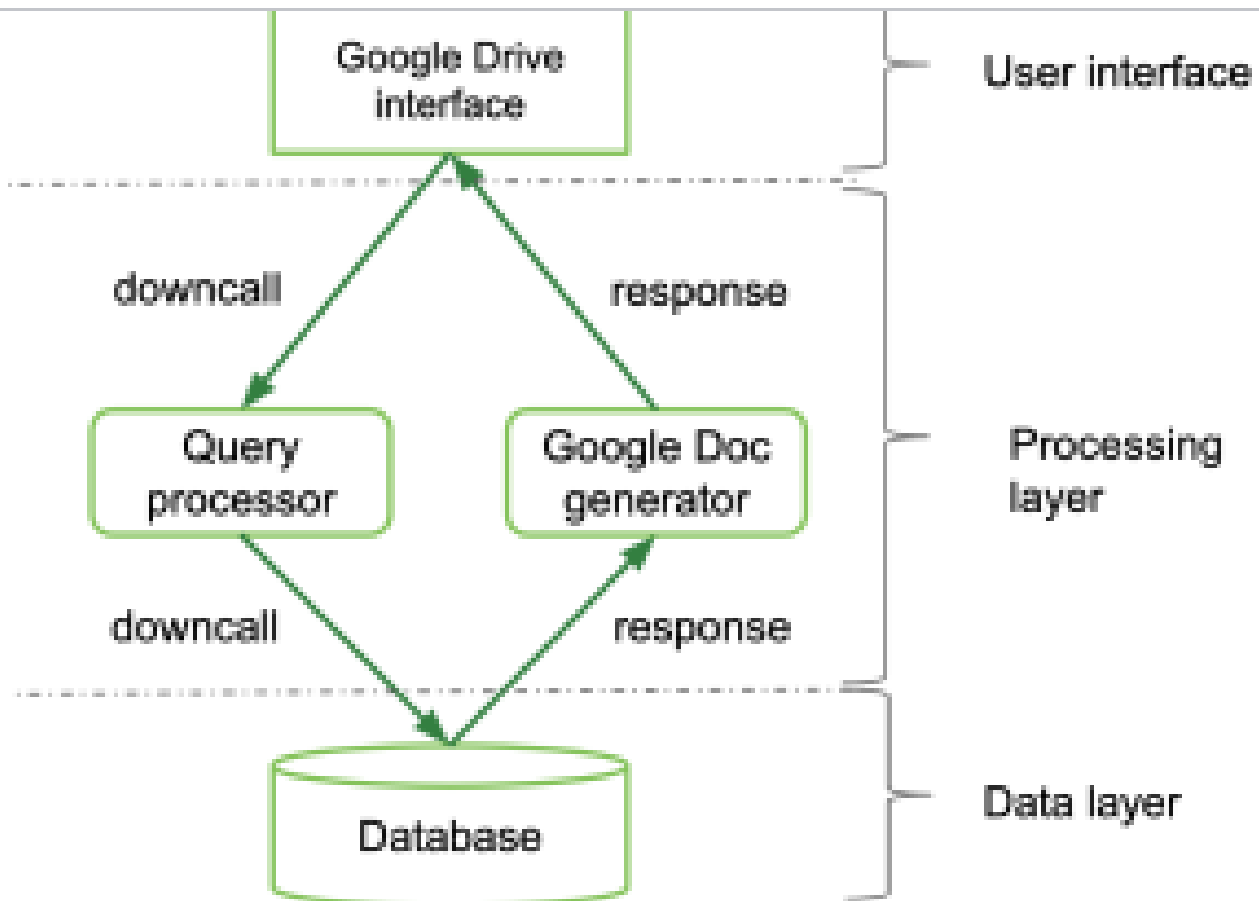
ТРЕНДОВЫЕ ИСТОРИИ

1. [Introduction to Cloud Native Computing](#)
2. [Microsoft wants to make service mesh invisible](#)
3. [Local or Cloud: Choosing the Right Dev Environment](#)
4. [Microservices 101](#)
5. [Tetrade launches open source marketplace to simplify Envoy adoption](#)

Рассмотрим в качестве примера Google Диск/Документы:

1. **Уровень интерфейса:** вы запрашиваете последний документ с вашего диска.
2. **Уровень обработки:** обрабатывает ваш запрос и запрашивает информацию на уровне данных.
3. **Уровень данных:** хранит постоянные данные (то есть ваш файл) и обеспечивает доступ к уровням более высокого порядка.

Уровень данных возвращает информацию на уровень обработки, который, в свою очередь, отправляет ее в интерфейс, где вы можете ее просматривать и редактировать. Несмотря на то, что это единый процесс, он разбит на три (или более) компонента на трех отдельных уровнях. Каждый уровень может находиться на отдельном компьютере, а может и не находиться (это зависит от архитектуры системы).

[Zoom](#)

Simplified organization of a Google Drive/Doc view request

2. Object-Oriented, Service-Oriented Architectures, Microservices, and Mesh Architectures

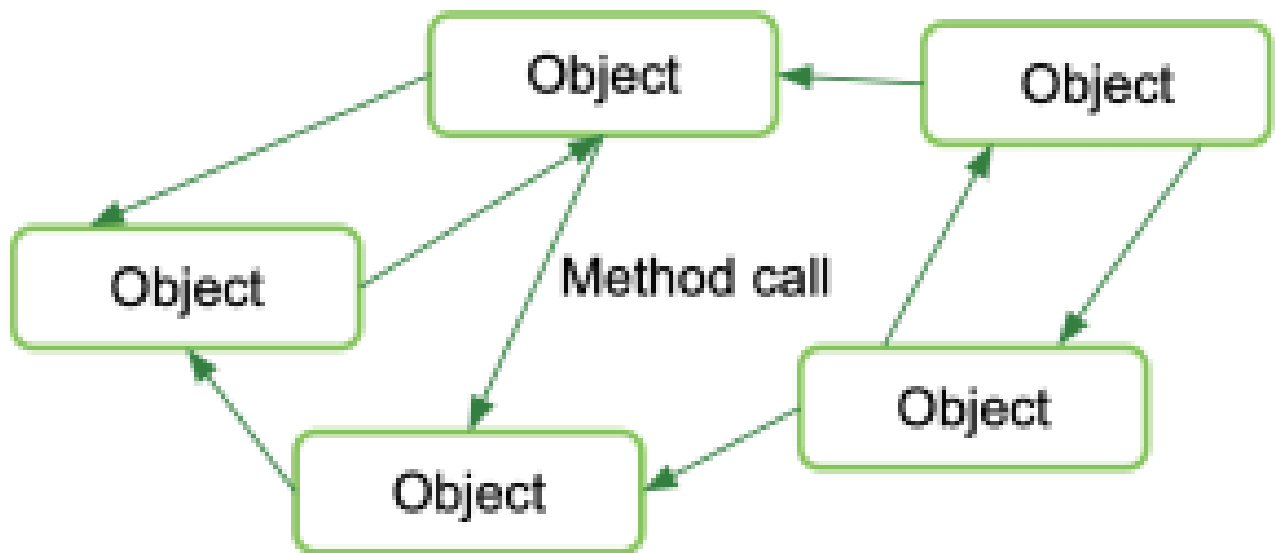
Object-oriented, service-oriented architectures (SOA), microservices, and “mesh” architectures are all more loosely organized and represent an evolutionary sequence. While we’ve grouped them together, object-oriented isn’t an architectural style but rather a programming methodology that makes service-oriented architectures (SOAs) and microservices possible.

Object-Based Architectural Styles

Object-oriented programming is a methodology generally used in the context of monolithic apps (although it’s also used in more modern architectures). Within the monolith, logical components are grouped together as objects. While they are distinguishable components, objects are still highly interconnected and not easy to separate. Object-oriented is a way to organize functionality and manage complexity within monoliths.

they store data. In this context, state stands for data. An object's method is the operations performed on that data.

Objects are connected through **procedure call mechanisms**. During a procedure call, an object “calls” on another object for specific requests. So when you hear “procedure call”, think of a request.



Zoom

Object-based architectural style

Service-Oriented Architecture

Objects form the foundation of encapsulating services into independent units leading to the development of SOAs. Services are self-contained, independent objects that make use of other services. Communication happens over a network via “messages” sent to each interface.

Microservices

Microservices are the next step in this evolutionary sequence. These microservices are smaller than services in an SOA, less tightly coupled, and more lightweight. The more significant difference from a business perspective, however, is their ability to decrease time to market.

microservices are more independent, allowing developers to push out updates without worrying about architectural details. Additionally, developers can use any programming language they want. Selecting the best language for a particular program further increases speed and thus time to market.

Mesh architectures

Mesh architectures are formed by services or processes running on nodes that cannot be easily accounted for. They may connect and disconnect frequently, some may not even use the internet. These services establish temporary peer-to-peer connections and can stay anonymous throughout the process. Examples are peer-to-peer technologies like TOR, torrents, p2p messengers, blockchain, etc.

Mesh architectures bring two additional qualities:

1. **Interacting services/processes are more uniform:** there may be just a few or even one type of service participating in a mesh network. They are considered equal to each other — equally trustworthy or untrustworthy if we are speaking about security, for instance. This is quite different from traditional service-based architectures where there are usually dozens of non-uniform services.
2. **There is a higher emphasis on its distributed nature.**

Mesh technologies are able to remain efficient even in highly unstable environments where connectivity between components is easily broken. Some components, in some cases even most components, are not directly connected. Instead, they communicate over multiple "hops" via other system elements (messages "hop" or "jump" from one element to another until reaching its destination).

Although you can see the fact that there is an evolution from object-oriented programming to SOAs, microservices, and mesh architectures, it doesn't mean that this methodology is obsolete. Object-oriented merely refers to the separation of blocks inside a component or monolith. It's a methodology that was, and still is, used. In fact, you can develop object-oriented microservices where microservices are composed of objects.

3. Representational State Transfer (REST) or RESTful Architectures

REST is an architectural style widely adopted for the Web (a Web-native architecture). A distributed system is viewed as a huge collection of resources,

- They understand a limited set of commands, often restricted to put, get, delete, and post
- There is only one (URL-based) naming scheme
- Resource interfaces follow the same conventions and semantics
- Messages are fully self-described
- After executing an operation on a resource, the resource forgets everything about the call from the client, except for the resulting changes in the service state

That last property is called **stateless execution**.

Why all these restrictions on commands and interface specs? Well, just think of the vastly diverse systems that must communicate over the internet. By standardizing interfaces and reducing commands, developing compatible systems is much easier. REST is basically a simplification to deal with the diversity in a huge distributed system like the internet.

Despite these restrictive commands, developers have enough flexibility to express anything they need. That's because the URL, a descriptive resource address, contains all the needed details to express operations. Let's take HTTP GET `http://amazon.com/shoppingcart/546/items/12345` as an example. This command is a request to get items "12345" from shopping cart "546." All the details about what it is and where to find it are expressed in the URL.

4. Publish-Subscribe Architectures

Publish-subscribe or, as a techie would say, pub/sub, is an even more loosely coupled architecture that allows processes to easily join or leave. The key difference here is how services communicate. Instead of calling and getting a response, services send one-way, usually asynchronous messages, generally not to a specific receiver. They rely on a configurator, administrator, or developer to configure who'll receive what message. In some cases, the receivers themselves can sign up to receive messages. This, by the way, is how you get your breaking news push notifications. *The Washington Post*, for instance, publishes a news item categorized as "breaking news" and whoever subscribes to these updates will receive it.

soon as you lose connectivity. With pub/sub, on the other hand, you'll simply get the update later (e.g. worst-case scenario, everyone else finds out about Trump's latest tweet before you do), but the user experience itself isn't affected.

System Architectures

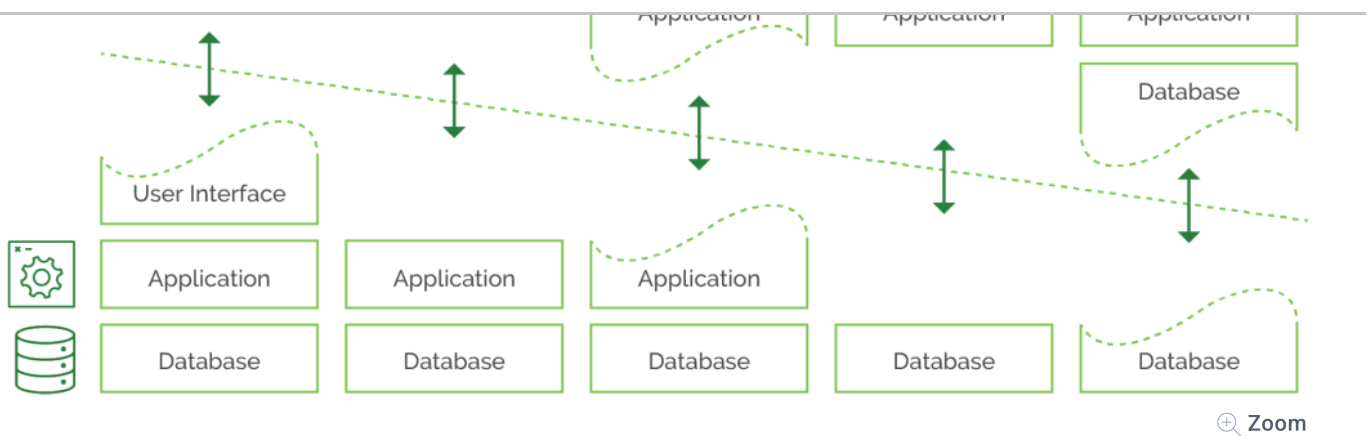
System architecture encompasses decisions as to where to place specific software components. Should certain components be placed on the same server or on different machines? Enterprises may have a specific high-speed processing server or high-end reliable storage facility that they'll want to leverage for specific components. These decisions will lead to different types of architectural organizations. We can broadly categorize them into centralized and decentralized organizations.

1. Centralized Organizations: Client-Server Systems

A server is a process implementing a specific service (e.g. database service). The client is a process requesting that service from a server. The client sends the request and waits for the reply (request-reply behavior). Where these processes are physically distributed will lead to different **multitiered architectural styles**.

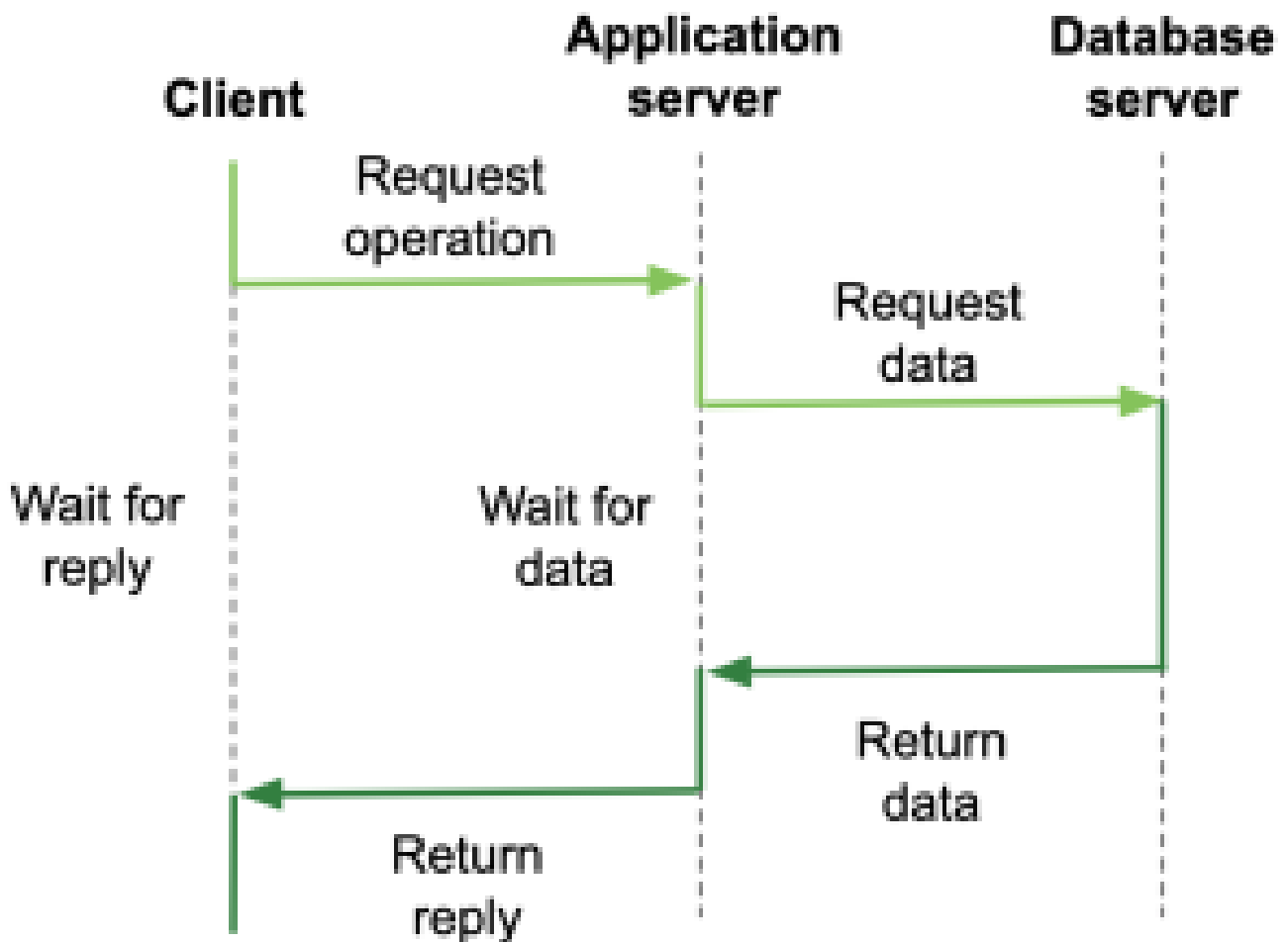
Remember the three layers discussed above (interface, processing, data layer)? While we distinguish between three logical levels, there are several ways to physically distribute a client-server application across machines.

The simplest organization is using two types of machines: 1) a client machine with the user interface (layer one), and 2) the server interface containing the program implementing the process and the data (layers two and three). In this case, everything is handled by the server while the client is nothing more than a dumb terminal. This is called a **(physically) two-tiered architecture**. However, there are other ways to separate these three layers on two machines. Possibilities range between separation, as mentioned above, to having everything, except the stored data, running on the client machine. You can see these different variations in the graphic below.



Variations of physically two-tiered three-layered architectures.

In (physically) three-tiered architectures, one of the servers also has to act as a client. The application is spread across three machines, one client and two servers. One of the servers may need input from the other server to process the client request, acting as a client.

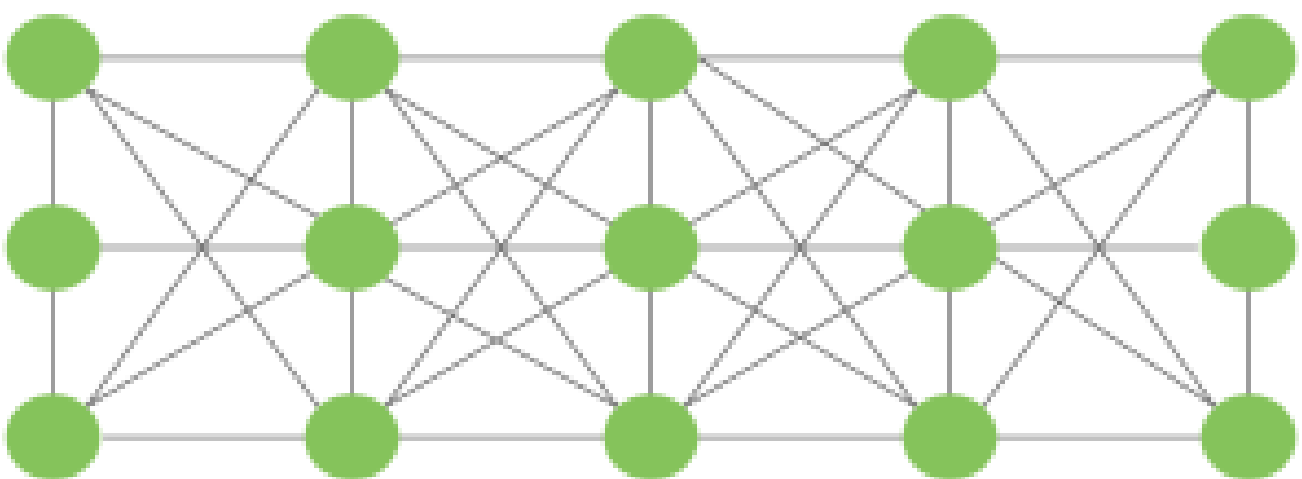


Multitiered client-server architectures are a direct consequence of dividing distributed applications into a user interface, processing components, and data-management components. Different tiers correspond directly with the logical organization of applications (software architecture corresponds to system architecture). This is called **vertical distribution** in which logically different components are placed on different machines. This allows for each machine to be tailored to a specific function, for example, placing the processing layer on a high-end processing server.

2. Decentralized Organization: Peer-to-Peer Systems

In modern architectures, clients and servers themselves are distributed, in so-called **horizontal distribution**. A client or server may be physically split, but each part operates on its own share of complete data thus balancing the load. So, instead of having one data store all components have access to, potentially turning into a bottleneck, each component has its own dataset. These are called **peer-to-peer systems**. Each process acts as client and server at the same time, often referred to as **servant**.

Peer-to-peer architectures evolve around the question of how to organize processes in an **overlay network** – a network in which nodes are formed by processes and links represent possible communication channels.



🔍 Zoom

Peer-to-peer system network connected via an overlay network.

As we've seen, there are multiple ways of organizing applications into logical components (software architecture considerations). Whether layered,

physical machines (system architecture considerations), either in a client-server or peer-to-peer fashion. In our article “**Primer: Understanding the Cloud Native Impact on Architecture**,” we discuss how cloud native is breaking the barriers of more traditional technologies, enabling enterprises to quickly adapt to market demands.

*As usual, a big thanks to **Oleg Chunikhin** who was instrumental in getting all technical details right. And to **Evgeny Pishnyuk** who also provided valuable feedback.*

TNS



Catherine Paganini aims to drive diversity and inclusion across the cloud native landscape. As CNCF TAG Contributor Strategy Co-chair and Deaf & Hard of Hearing WG facilitator, she founded and/or facilitates multiple inclusion initiatives (BIPOC, blind/visually impaired, LGBTQ+, stuttering/speech). Her...

[Read more from Catherine Paganini →](#)

SHARE THIS STORY



TRENDING STORIES

1. **Introduction to Cloud Native Computing**
2. **Microsoft wants to make service mesh invisible**
3. **Local or Cloud: Choosing the Right Dev Environment**
4. **Microservices 101**
5. **Tetrade launches open source marketplace to simplify Envoy adoption**

INSIGHTS FROM OUR SPONSORS

How to Scrape Bing Reverse Image Search Results

22 June 2026

Measuring Brand Presence Across AI Answer Engines

22 June 2026

SerpApi Weekly Changelog: June 15-21, 2026

22 June 2026



Open Governance for MySQL: A Step Forward for the Community

25 June 2026

Governing AI Assets at Scale with MCP Gateway and Registry

17 June 2026

Introducing Trusted Remote Execution: Policy-Enforced Scripts for AI Agents and Humans

4 May 2026



Distributed Databases: An Overview

16 June 2026

On-Device AI: Benefits, Use Cases, and Challenges

12 June 2026

Accelerating AI in Healthcare: Fix Data Infrastructure Before AI Fails Become a Board Priority

11 June 2026

most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)
[Sponsors](#)
[Advertise With Us](#)
[Contributions](#)



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)
[Backend Developer Roadmap](#)
[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)
[Terms of Use](#)
[Advertising Terms & Conditions](#)
[Privacy Policy](#)
[Cookie Policy](#)